Using virtual environments

Each time you install a package in Python, you're making it available to the entire system. As different projects have different needs, this can cause conflicts. To deal with this, Python uses *virtual environments*, which are self-contained Python setups that are isolated from each other. It is important to use a virtual environment when writing Python to avoid packaging conflicts.

Appendix A contains a brief overview of installing packages in Python virtual environments. For a full explanation on the technology, see http://mng.bz/JZVZ.

Each Django project is a Python program, but because it is built using a framework, the project provides the execution paths. When you create a new project, Django installs the bare minimum you need inside a new directory with the name that you give it.

At the heart of a Django project is a file named wsgi.py. WSGI, a Python specification that describes how code interacts with a web server, stands for *Web Server Gateway Interface*. You really don't need to know much about this, especially when you're working in your development environment; the file gets generated for you. Inside wsgi.py is a short program responsible for interfacing between your code and the web server. When you're ready to put your project in production, you'll configure your web server to point at this application in your project. This is why Django is considered a framework rather than a library: wsgi.py is the entry point, and most of the code you write will be called through it. Figure 2.1 shows you how the WSGI protocol acts as a bridge to your code through the wsgi.py file.

When you build RiffMates, or any other website, what you're really trying to do is make web pages. Each page lives at a URL, and each time a user visits a URL, you're going to want some code called to generate the page. In Django, this code is called a *view*.

When you load a page, or in some cases interact with one, your web browser is communicating with a web server. In the case of a Django project, the web server communicates with the WSGI application defined in wsgi.py. The WSGI application triggers the Django framework code. Django runs a view that is responsible for returning content that Django turns into an HTTP response for the browser.

Django needs to know what view to call for each URL on the website. This is done by mapping URLs to views. Within the framework, a URL sometimes is known as a *route*. Each Django project has a main urls.py file that contains a list of URL mappings, called `urlpatterns`. The list contains `django.urls.path` objects that specify a pattern that matches one or more URLs and what view to call when that URL gets visited.

The wsgi.py and urls.py files are key to your Django project. Every new website you build with Django requires its own copy of these files. To make your life easier, the `django-admin` command that comes with the package creates these files for you, customizing them based on your project name.